

Distributed Systems

Lecture21: Replication

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Why Replicate!

- Assume a simple model in which we make a copy of a specific part of a system (meaning code and data).
  - Increase reliability: if one copy does not live up to specifications, switch over to the other copy while repairing the failing one.
  - Performance: simply spread requests between different replicated parts to keep load balanced, or to ensure quick responses by taking proximity into account.
- **The problem!**
  - Having multiple copies, means that when any copy changes, that change should be made at all copies: **replicas need to be kept the same**, that is, be kept **consistent**.

Performance and Scalability

- **Main Issue**
  - To keep replicas consistent, we generally need to ensure that all conflicting operations are done in the same order everywhere
- Conflicting operations: From the world of transactions
  - Read–write conflict: a read operation and a write operation act concurrently
  - Write–write conflict: two concurrent write operations
- **Issue:** Guaranteeing global ordering on conflicting operations may be a costly operation, downgrading scalability.
- **Solution:** weaken consistency requirements so that hopefully global synchronization can be avoided

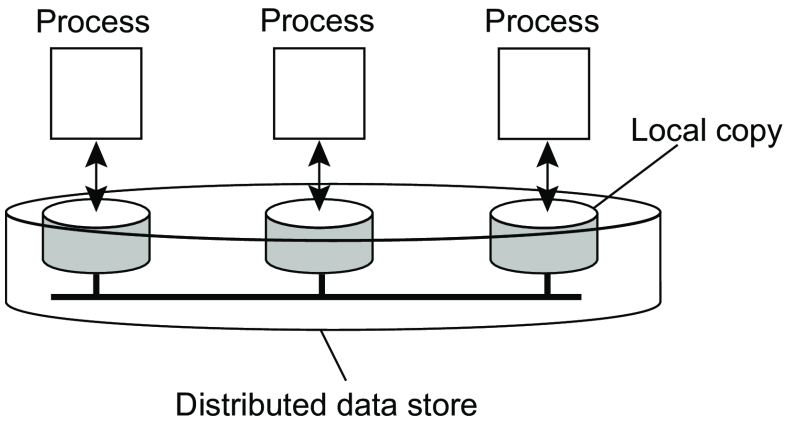
Data-centric Consistency Models

- **Consistency model**

  A contract between a (distributed) data store and processes, in which the data store specifies precisely what the results of read and write operations are in the presence of concurrency.

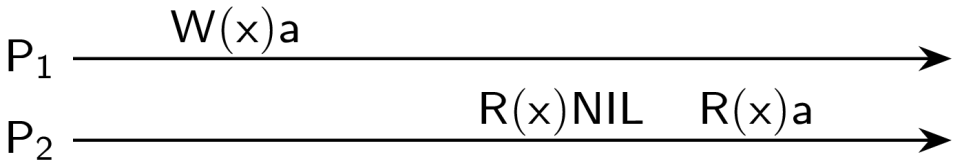
- **Essential**

  A data store is a distributed collection of storages:



Some Notations

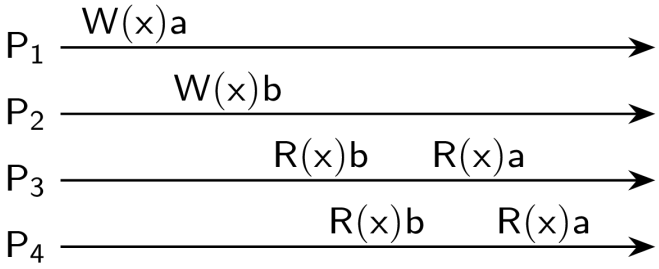
- **Read and write operations**
  - $W_i(x)a$: Process P_i writes value a to x
  - $R_i(x)b$: Process $P_i$reads value b to x
  - All data items initially have value NIL
- **Possible behavior**
  - We omit the index when possible and draw according to time (x-axis):



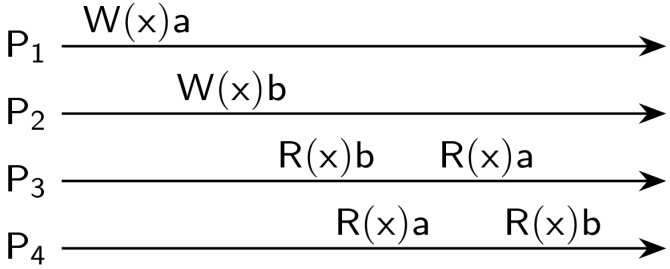
Sequential Consistency

- **Definition**

  The result of any execution is the same as if the operations of all processes were executed in some sequential order, and the operations of each individual process appear in this sequence in the order specified by its program.



A Sequentially Consistent Data Store



A Data Store That is not Sequentially Consistent

Example

- **Three concurrent processes (initial values: 0)**

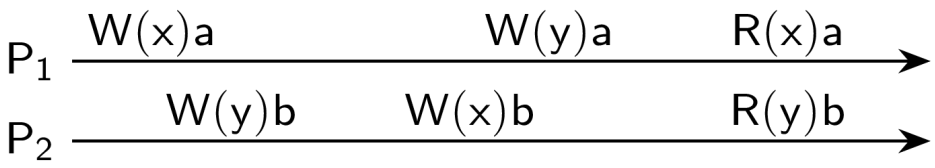
Process P_1	Process P_2	Process P_3
`x ← 1;`	`y ← 1;`	`z ← 1;`
`print(y,z);`	`print(x,z);`	`print(x,y);`

- **Example execution sequences**

Execution 1	Execution 2	Execution 3	Execution 4
P_1: `x ← 1;`	P_1: `x ← 1;`	P_2: `y ← 1;`	P_2: `y ← 1;`
P_1: `print(y,z);`	P_2: `y ← 1;`	P_3: `z ← 1;`	P_1: `x ← 1;`
P_2: `y ← 1;`	P_2: `print(x,z);`	P_3: `print(x,y);`	P_3: `z ← 1;`
P_2: `print(x,z);`	P_1: `print(y,z);`	P_2: `print(x,z);`	P_2: `print(x,z);`
P_3: `z ← 1;`	P_3: `z ← 1;`	P_1: `x ← 1;`	P_1: `print(y,z);`
P_3: `print(x,y);`	P_3: `print(x,y);`	P_1: `print(y,z);`	P_3: `print(x,y);`
Prints: `001011`	*Prints:* `101011`	*Prints:* `010111`	*Prints:* `111111`
Signature: 0 0 1 0 1 1	*Signature:* 1 0 1 0 1 1	*Signature:* 1 1 0 1 0 1	*Signature:* 1 1 1 1 1 1
(a)	(b)	(c)	(d)

How Tricky Can It Get?

- **Seemingly okay**



- **But not really (don't forget that P1 and P2 act concurrently)**

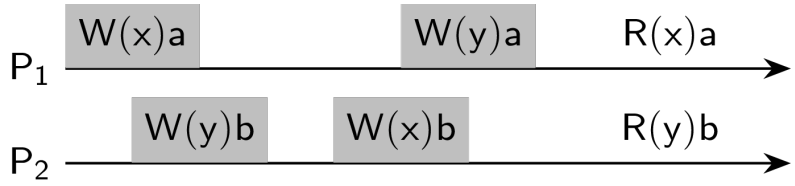
Possible ordering of operations	Result	
$W_1(x)a$; $W_1(y)a$; $W_2(y)b$; $W_2(x)b$	$R_1(x)b$	$R_2(y)b$
$W_1(x)a$; $W_2(y)b$; $W_1(y)a$; $W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_1(x)a$; $W_2(y)b$; $W_2(x)b$; $W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_1(x)a$; $W_1(y)a$; $W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_1(x)a$; $W_2(x)b$; $W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b$; $W_2(x)b$; $W_1(x)a$; $W_1(y)a$	$R_1(x)a$	$R_2(y)a$

How Tricky Can It Get?

- **Linearizability**

  Each operation should appear to take effect instantaneously at some moment between its start and completion.

- **Operations complete within a given time (shaded area)**



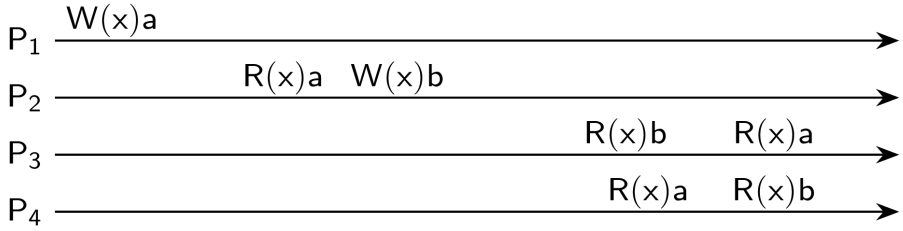
- **With better results**

Possible ordering of operations	Result	
$W_1(x)a; W_2(y)b; W_1(y)a; W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_1(x)a; W_2(y)b; W_2(x)b; W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b; W_1(x)a; W_1(y)a; W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b; W_1(x)a; W_2(x)b; W_1(y)a$	$R_1(x)b$	$R_2(y)a$

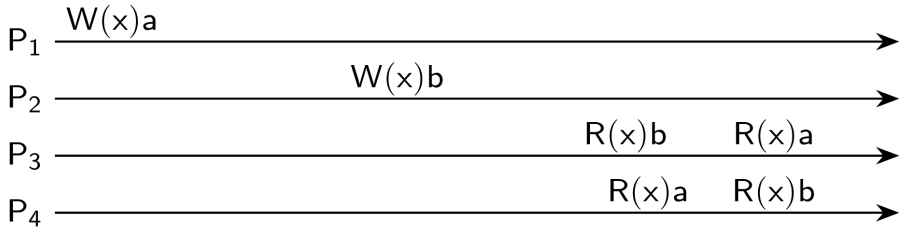
Causal Consistency

- **Definition**

  Writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order by different processes.



A violation of a causally-consistent store



A correct sequence of events in a causally-consistent store

Consistency Models, Serializability, Transactions

- Overwhelming, but often already known
- Again, from the world of transactions: can we order the execution of all operations in a set of transactions in such a way that the final result matches a serial execution of those transactions? The keyword is serializability.

```
BEGIN_TRANSACTION        BEGIN_TRANSACTION        BEGIN_TRANSACTION
  x = 0                    x = 0                    x = 0
  x = x + 1                x = x + 2                x = x + 3
END_TRANSACTION          END_TRANSACTION          END_TRANSACTION
```

Transaction T_1 Transaction T_2 Transaction T_3

- A number of schedules

	Time $\longrightarrow$						
S1	x = 0	x = x + 1	x = 0	x = x + 2	x = 0	x = x + 3	Legal
S2	x = 0	x = 0	x = x + 1	x = x + 2	x = 0	x = x + 3	Legal
S3	x = 0	x = 0	x = x + 1	x = 0	x = x + 2	x = x + 3	Illegal
S4	x = 0	x = 0	x = x + 3	x = 0	x = x + 1	x = x + 2	Illegal

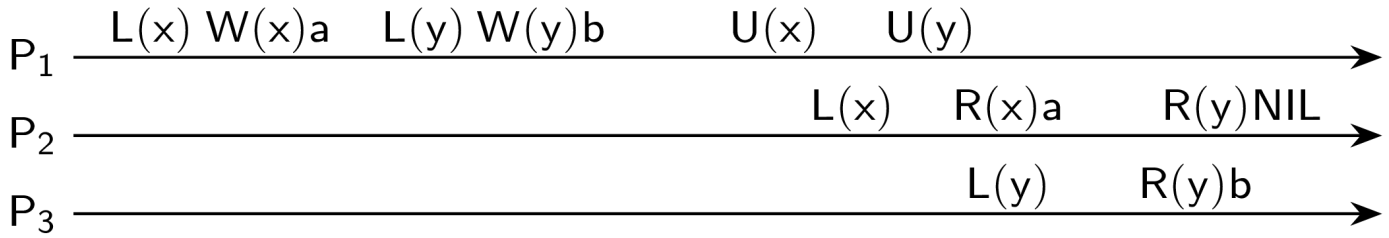
Grouping Operations

- **Entry consistency: Definition**
  - Accesses to locks are sequentially consistent.
  - No access to a lock is allowed to be performed until all previous writes have completed everywhere.
  - No data access is allowed to be performed until all previous accesses to locks have been performed.
- **Basic Idea**

  You don't care that reads and writes of a series of operations are immediately known to other processes. You just want the effect of the series itself to be known.

Grouping Operations

- A valid event sequence for entry consistency



- **Observation**

  Entry consistency implies that we need to lock and unlock data (implicitly or not).

- **Question**

  What would be a convenient way of making this consistency more or less transparent to programmers?

Eventual Consistency

- **Definition**

  Consider a collection of data stores and (concurrent) write operations. The stores are eventually consistent when in lack of updates from a certain moment, all updates to that point are propagated in such a way that replicas will have the same data stored (until updates are accepted again).

- **Strong eventual consistency**

  Basic idea: if there are conflicting updates, have a globally determined resolution mechanism (for example, using NTP, simply let the “most recent” update win).

- **Program consistency**

  P is a monotonic problem if for any input sets S and T , $P(S) \subseteq P(T)$. Observation: A program solving a monotonic problem can start with incomplete information, but is guaranteed not to have to roll back when missing information becomes available. Example: filling a shopping cart.

- **Important observation**

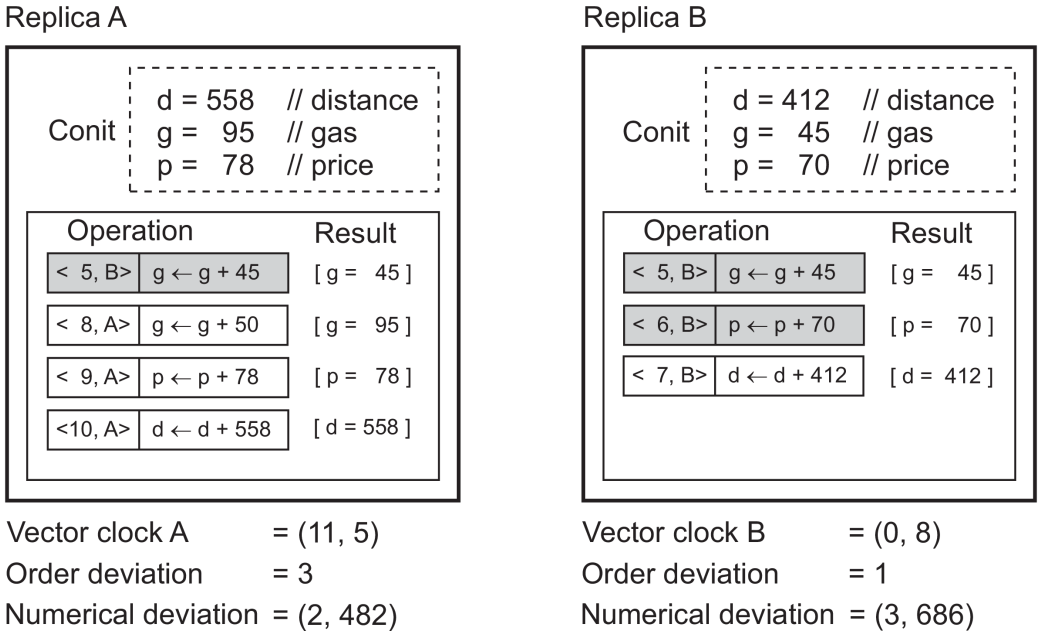
  In all cases, we are avoiding global synchronization.

Continuous Consistency

- **We can actually talk about a degree of consistency**
  - replicas may differ in their numerical value
  - replicas may differ in their relative staleness
  - there may be differences regarding (number and order) of performed update operations
- **Conit**

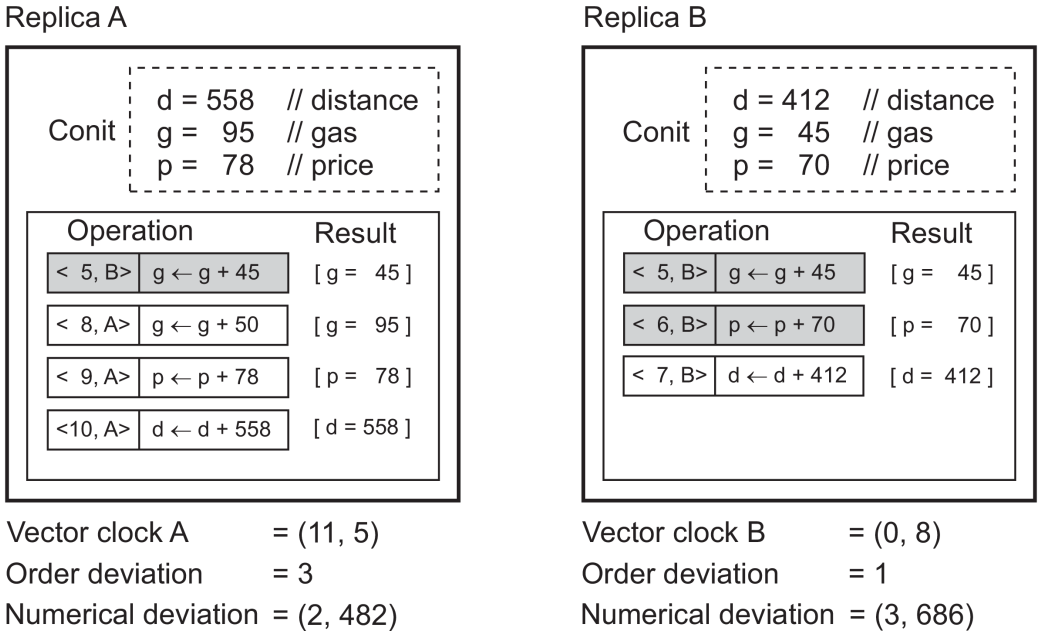
  Consistency unit ⇒ specifies the data unit over which consistency is to be measured.

Example: Conit



- **Conit (contains the variables g, p, and d)**
  - Each replica has a vector clock: ([known] time @ A, [known] time @ B)
  - B sends A operation [⟨5, B⟩ : g ← d + 45]; A has made this operation permanent (cannot be rolled back)

Example: Conit



- **Conit (contains the variables g, p, and d)**
  - A has three pending operations ⇒ order deviation = 3
  - A missed two operations from B; max diff is 70 + 412 units ⇒ (2, 482)

Consistency For Mobile Users

- **Example**

  Consider a distributed database to which you have access through your notebook. Assume your notebook acts as a front end to the database.

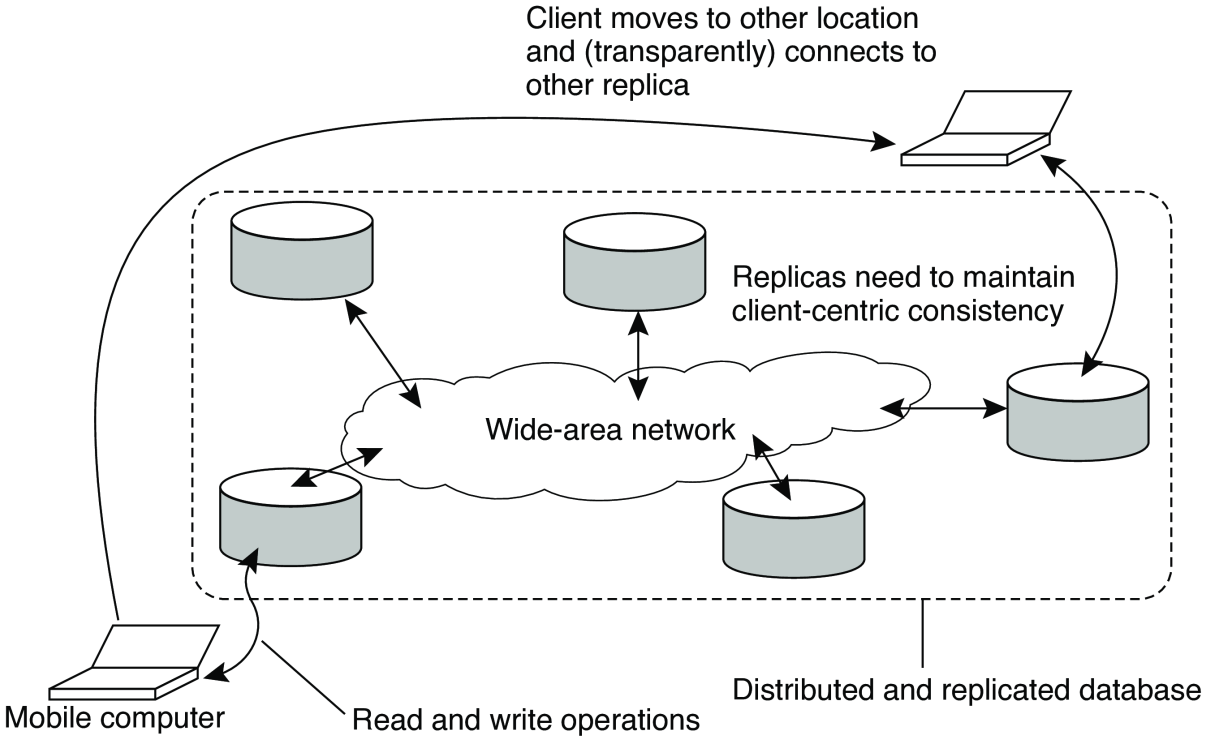
  - At location A you access the database doing reads and updates.
  - At location B you continue your work, but unless you access the same server as the one at location A, you may detect inconsistencies:
    - your updates at A may not have yet been propagated to B
    - you may be reading newer entries than the ones available at A
    - your updates at B may eventually conflict with those at A

- **Note**

  The only thing you really want is that the entries you updated and/or read at A, are in B the way you left them in A. In that case, the database will appear to be consistent to you.

Basic Architecture

- The principle of a mobile user accessing different replicas of a distributed database



Client-centric Consistency: Notation

- Notations
  - $W_1(x_2)$ is the write operation by process P_1 that leads to version x_2 of x
  - $W_1(x_i; x_j)$ indicates P_1 produces version x_j based on a previous version x_i.
  - $W_1(x_i | x_j)$ indicates P_1 produces version x_j concurrently to version x_i.

Monotonic Reads

- **Example**

  Automatically reading your personal calendar updates from different servers. Monotonic reads guarantees that the user sees all updates, no matter from which server the automatic reading takes place.

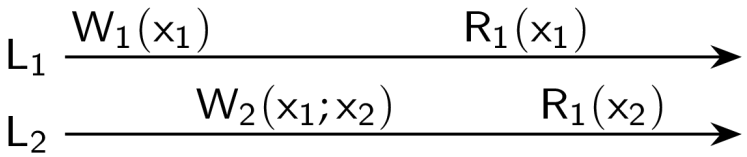
- **Example**

  Reading (not modifying) incoming mail while you are on the move. Each time you connect to a different e-mail server, that server fetches (at least) all the updates from the server you previously visited.

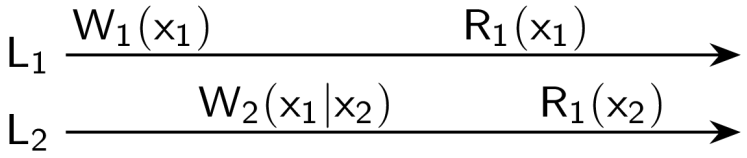
Monotonic Reads

- **Definition**

  If a process reads the value of a data item x , any successive read operation on x by that process will always return that same or a more recent value.



A monotonic-read consistent data store



A data store that does not provide monotonic reads

Monotonic Writes

- **Example**

  Updating a program at server S2 , and ensuring that all components on which compilation and linking depends, are also placed at S2 .

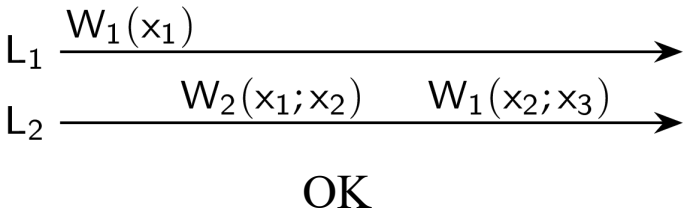
- **Example**

  Maintaining versions of replicated files in the correct order everywhere (propagate the previous version to the server where the newest version is installed).

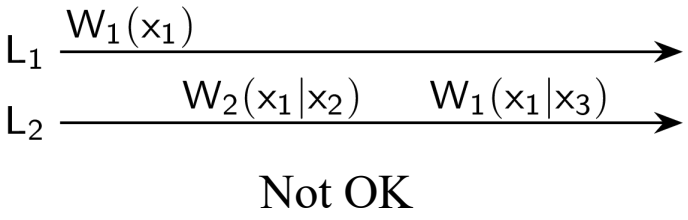
Monotonic Writes

- **Definition**

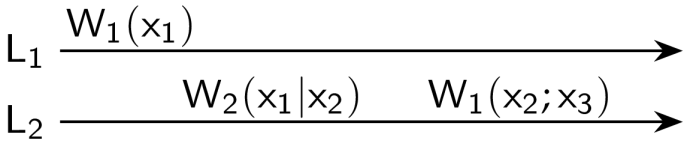
  A write operation by a process on a data item x is completed before any successive write operation on x by the same process.



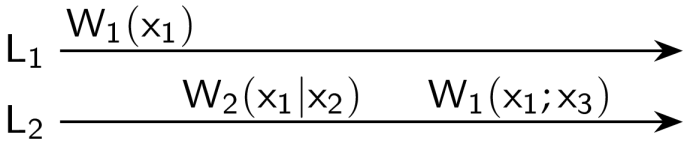
OK



Not OK



Not OK

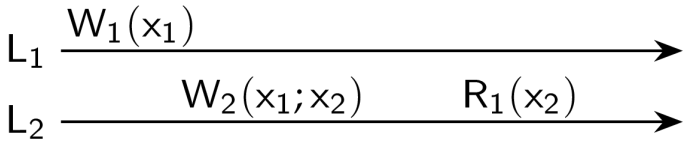


OK

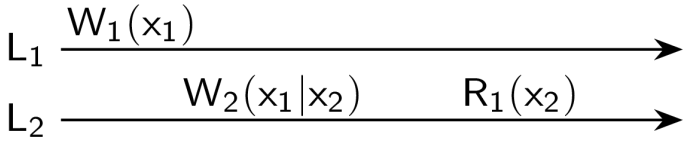
Read your Writes

- **Definition**

  The effect of a write operation by a process on a data item x , will always be seen by a successive read operation on x by the same process.



OK



Not OK

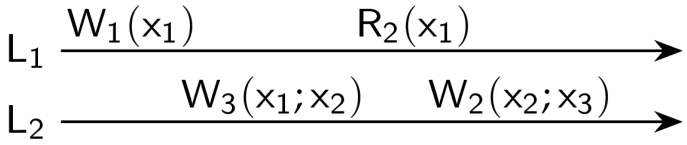
- **Example**

  Updating your Web page and guaranteeing that your Web browser shows the newest version instead of its cached copy.

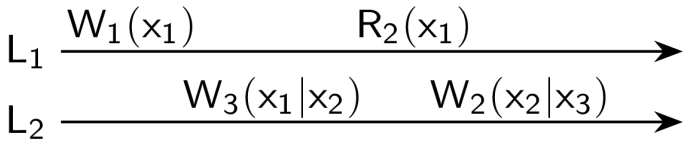
Writes follow Reads

- **Definition**

  A write operation by a process on a data item x following a previous read operation on x by the same process, is guaranteed to take place on the same or a more recent value of x that was read.



OK



Not OK

- **Example**

  See reactions to posted articles only if you have the original posting (a read “pulls in” the corresponding write operation).

Example: ZooKeeper consistency

- ZooKeeper's consistency model mixes elements of data-centric and client-centric models
- Take a naive example

